

Todo List Manager:

- Users can register and log in.
- Each user can create multiple todo lists.
- Within each todo list, users can add, edit, and delete tasks.
- Tasks can have a title, description, due date, and status (completed or pending).

```
CREATE TABLE users (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    username TEXT UNIQUE NOT NULL,  
    password_hash TEXT NOT NULL  
);  
  
CREATE TABLE todo_lists (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    user_id INTEGER NOT NULL,  
    title TEXT NOT NULL,  
    FOREIGN KEY (user_id) REFERENCES users (id)  
);  
  
CREATE TABLE tasks (  
    id INTEGER PRIMARY KEY AUTOINCREMENT,  
    todo_list_id INTEGER NOT NULL,  
    title TEXT NOT NULL,  
    description TEXT,  
    due_date DATE,  
    status TEXT NOT NULL CHECK (status IN ('pending', 'completed')),  
    FOREIGN KEY (todo_list_id) REFERENCES todo_lists (id)  
);
```

CODE

```
from fastapi import FastAPI, HTTPException, Depends, status  
from fastapi.security import OAuth2PasswordBearer, OAuth2PasswordRequestForm  
from pydantic import BaseModel  
from passlib.context import CryptContext  
from typing import List, Optional  
from datetime import date  
from sqlalchemy import create_engine, Column, Integer, String, Date, ForeignKey  
from sqlalchemy.orm import sessionmaker, relationship  
from sqlalchemy.ext.declarative import declarative_base  
  
# SQLAlchemy models  
Base = declarative_base()  
  
class User(Base):
```

```

__tablename__ = "users"
id = Column(Integer, primary_key=True, index=True)
username = Column(String, unique=True, index=True)
password_hash = Column(String)

todo_lists = relationship("ToDoList", back_populates="owner")

class ToDoList(Base):
    __tablename__ = "todo_lists"
    id = Column(Integer, primary_key=True, index=True)
    title = Column(String, index=True)
    user_id = Column(Integer, ForeignKey("users.id"))

    owner = relationship("User", back_populates="todo_lists")
    tasks = relationship("Task", back_populates="todo_list")

class Task(Base):
    __tablename__ = "tasks"
    id = Column(Integer, primary_key=True, index=True)
    title = Column(String, index=True)
    description = Column(String, nullable=True)
    due_date = Column(Date, nullable=True)
    status = Column(String, index=True)
    todo_list_id = Column(Integer, ForeignKey("todo_lists.id"))

    todo_list = relationship("ToDoList", back_populates="tasks")

# Create SQLite database engine and session
SQLALCHEMY_DATABASE_URL = "sqlite:///./todo.db"
engine = create_engine(SQLALCHEMY_DATABASE_URL)
SessionLocal = sessionmaker(autocommit=False, autoflush=False, bind=engine)

# FastAPI app
app = FastAPI()

# Password hashing
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

# Dependency to get database session
def get_db():
    db = SessionLocal()
    try:
        yield db
    finally:
        db.close()

# Pydantic models
class ToDoListBase(BaseModel):
    title: str

class TaskBase(BaseModel):
    title: str

```

```
description: Optional[str] = None
due_date: Optional[date] = None
status: str = "pending"
```

```
class TodoListCreate(TodoListBase):
    pass
```

```
class TaskCreate(TaskBase):
    pass
```

```
class TodoList(TodoListBase):
    id: int
    tasks: List[TaskBase] = []
```

```
class Config:
    orm_mode = True
```

```
class Task(TaskBase):
    id: int
    todo_list_id: int

    class Config:
        orm_mode = True
```

```
# Routes
```

```
@app.post("/users/", response_model=User)
```

```
def create_user(username: str, password: str, db: Session = Depends(get_db)):
    hashed_password = pwd_context.hash(password)
    user = User(username=username, password_hash=hashed_password)
    db.add(user)
    db.commit()
    db.refresh(user)
    return user
```

```
@app.post("/todo_lists/", response_model=TodoList)
```

```
def create_todo_list(todo_list: TodoListCreate, db: Session = Depends(get_db)):
    db_todo_list = TodoList(**todo_list.dict())
    db.add(db_todo_list)
    db.commit()
    db.refresh(db_todo_list)
    return db_todo_list
```

```
@app.get("/todo_lists/{todo_list_id}", response_model=TodoList)
```

```
def read_todo_list(todo_list_id: int, db: Session = Depends(get_db)):
    todo_list = db.query(TodoList).filter(TodoList.id == todo_list_id).first()
    if todo_list is None:
        raise HTTPException(status_code=404, detail="Todo list not found")
    return todo_list
```

```
@app.post("/todo_lists/{todo_list_id}/tasks/", response_model=Task)
```

```
def create_task_for_todo_list(todo_list_id: int, task: TaskCreate, db: Session = Depends(get_db)):
    db_task = Task(**task.dict(), todo_list_id=todo_list_id)
    db.add(db_task)
```

```
db.commit()
db.refresh(db_task)
return db_task

@app.get("/todo_lists/{todo_list_id}/tasks/", response_model=List[Task])
def read_tasks(todo_list_id: int, db: Session = Depends(get_db)):
    tasks = db.query(Task).filter(Task.todo_list_id == todo_list_id).all()
    return tasks
```

This code provides basic CRUD operations for users, todo lists, and tasks. Users can register, create todo lists, add tasks to todo lists, and retrieve tasks for a specific todo list. Make sure to install required packages such as fastapi, uvicorn, passlib, pydantic, sqlalchemy, and sqlite before running this code.